

Universidade de Coimbra

LECD

Implementação de inteligência artificial no jogo da cobra

Tópicos de Inteligência Artificial

Aluno: Tiago Carlos 2021218666

Aluno: Lucas Saraiva 2021218251

Introdução

Nos dias de hoje, o impacto da inteligência artificial (IA) no mundo que nos rodeia é cada vez mais evidente, com a utilização de sistemas baseados em IA a crescer de forma drástica. Neste projeto, decidimos explorar um modelo de IA num contexto relativamente simples e intuitivo: o jogo da cobra, criado nos anos 70. O objetivo principal do nosso trabalho é desenvolver um programa que automatize a jogabilidade da cobra, permitindo que esta evite colisões e consiga apanhar o maior número de frutas possível.

Para alcançar este objetivo, utilizamos técnicas de **Reinforcement Learning**. Estas técnicas são amplamente aplicáveis a vários modelos e agentes, sendo o jogo da cobra uma plataforma didática ideal para a sua implementação. Além disso, o jogo oferece uma forma visual e prática de compreender como estas técnicas podem ser aplicadas em diferentes problemas e contextos de inteligência artificial.

Problemas semelhantes, como a automatização de jogadores em jogos virtuais, têm-se tornado cada vez mais populares. Muitas vezes, esses jogos são abstrações ou virtualizações de cenários reais. No caso do jogo da cobra, podemos fazer uma analogia com um aspirador automático, onde as frutas representam o lixo, e os limites do ecrã simulam as paredes da divisão. Com os avanços na área da inteligência artificial, problemas desta natureza podem ser resolvidos através de diversas abordagens. Em particular, o **Reinforcement Learning** tem-se destacado, sendo amplamente utilizado em jogos como o xadrez e outros cenários complexos.

As abordagens tradicionais, como os métodos baseados em regras, enfrentam limitações significativas, sobretudo em ambientes mais complexos, que envolvem movimentos imprevisíveis ou espaços limitados. Além disso, muitos desses métodos não são suficientemente eficientes para treinar modelos de IA em tempo real, o que restringe a sua aplicabilidade prática.

Neste trabalho, adotamos uma abordagem baseada no algoritmo **Q-Learning** para treinar a nossa cobra e permitir a sua automatização. Este algoritmo permite que um agente aprenda a tomar decisões num determinado ambiente, com o objetivo de maximizar uma recompensa a longo prazo. O processo funciona através da interação do agente com o ambiente, onde este realiza ações e recebe recompensas ou penalizações. Com base nessas interações, o agente atualiza uma tabela Q, que armazena os valores esperados de cada ação em cada estado. A ideia central é que, ao optar sempre pela ação com o maior valor Q em cada estado, o agente aprende a tomar as melhores decisões para atingir os seus objetivos.

Em resumo, o Q-Learning é um método de aprendizagem por tentativa e erro, onde o agente aprende a maximizar recompensas futuras através da exploração e exploração de ações. Embora tenhamos aplicado este algoritmo num ambiente simples, a sua eficiência e

simplicidade tornam-no numa das abordagens mais adequadas e otimizadas para resolver problemas deste tipo.

Conceitos Fundamentais, Teoria do projeto, State of the Art

Reinforcement Learning (RL)

O **Reinforcement Learning (RL)** é uma área importante da inteligência artificial que se foca em ensinar algoritmos a tomar decisões em ambientes dinâmicos com o objetivo de maximizar uma recompensa cumulativa. Em termos simples, o RL permite que um agente aprenda a interagir com o ambiente através de recompensas e penalizações, melhorando o seu desempenho ao longo do tempo.

Neste tipo de projeto, os principais componentes a implementar são:

- **Agente:** A entidade (neste caso, a "cobra") que é treinada para interagir com o ambiente.
- **Ambiente:** O "mundo" onde o agente opera e que fornece informações sobre o estado e as recompensas.
- **Estado:** Representa o momento atual do ambiente.
- **Ações:** As opções que o agente pode tomar num estado específico.
- **Recompensa:** Um feedback numérico que informa o agente sobre a qualidade da sua ação.
- **Política:** A estratégia usada pelo agente para selecionar ações em estados definidos. Pode ser determinística ou probabilística.
- **Função de valor:** Mede a qualidade de um estado em termos das recompensas futuras esperadas.
- **Modelo:** Uma representação do ambiente que pode ser usada por métodos que tentam prever resultados sem interação direta.

O modelo segue um ciclo de treino:

1. O agente observa o estado atual.
2. Escolhe uma ação com base na política.
3. Realiza a ação, obtém uma recompensa e observa o próximo estado.
4. Atualiza a política ou a função de valor para melhorar as suas decisões futuras.

Modelo Utilizado

O modelo de aprendizagem que utilizámos foi uma **Deep Q-Network (DQN)**, um algoritmo de Reinforcement Learning que combina o **Q-Learning** com uma **Deep Neural Network (DNN)**.

Q-Learning

O **Q-Learning** é um algoritmo cujo objetivo é aprender a função $Q(s,a)$, que estima o valor esperado de realizar uma ação a num estado s , considerando as recompensas futuras. A equação de atualização é dada por: $Q(s, a) = Q(s, a) + \alpha * (r + \gamma * \max_{a'} Q(s', a') - Q(s, a))$

- $Q(s,a)$: Valor atual do par estado-ação (s,a) .
- r : Recompensa imediata recebida após executar a ação a no estado s .
- α : Taxa de aprendizagem (valores entre 0 e 1).
- γ : Fator de desconto (valores entre 0 e 1) que pondera a importância das recompensas futuras.
- $\max_{a'} Q(s', a')$: Valor máximo dos Q-valores para todas as ações possíveis no próximo estado, s' .

No entanto, armazenar $Q(s,a)$ como uma tabela torna-se inviável em ambientes com muitos estados e ações. A integração do Q-Learning com uma **Deep Neural Network** resolve este problema.

Deep Neural Networks (DNN)

Uma **Deep Neural Network** é composta por:

- **Input layer**: Recebe as variáveis de entrada.
- **Hidden layers**: Camadas intermédias que processam os dados, aplicando combinações lineares ($Wx+b$) e funções de ativação para introduzir não linearidades.
- **Output layer**: Fornece a previsão final.

Durante o treino, os pesos W e os bias b são ajustados para minimizar o erro entre a previsão e o valor real, utilizando os algoritmos de **backpropagation** e **gradient descent**.

Deep Q-Network (DQN)

Uma DNN torna-se numa DQN ao ser usada juntamente com o algoritmo **Q-Learning**. Em vez de uma tabela de valores $Q(s,a)$, é utilizada uma rede neural que estima os valores Q para cada estado s e ação a , permitindo lidar com ambientes de alta dimensionalidade.

- **Input layer:** Recebe apenas o estado s .
 - **Output layer:** Devolve os valores $Q(s,a)$ para todas as ações possíveis. Por exemplo, se houver 4 ações possíveis, a saída será um vetor com 4 valores Q , cada um associado a uma ação.
 - **Hidden layers:** Utilizam funções de ativação como ReLU, semelhantes às DNN tradicionais.
-

Treino do Modelo

Durante o treino, o agente utiliza o algoritmo **ϵ -Greedy**:

- Com probabilidade ϵ , escolhe uma ação aleatória (exploração).
- Com probabilidade $1-\epsilon$, escolhe a ação com o maior valor $Q(s,a)$ (exploração).
Inicialmente, ϵ é alto para incentivar a exploração, mas reduz gradualmente para favorecer decisões mais informadas.

As interações (s,a,r,s') são armazenadas num **buffer de replay**, que é mostrado aleatoriamente durante o treino. Este método reduz a correlação entre amostras consecutivas, melhorando a estabilidade do treino.

Redes na DQN

A DQN utiliza duas redes neurais:

1. **Rede Principal:** Atualizada frequentemente é usada para escolher as ações.
2. **Rede Alvo:** Atualizada periodicamente com os pesos da rede principal é usada para calcular os valores Q_{target} . Esta abordagem estabiliza o treino, mantendo os valores-alvo constantes por algumas iterações.

Atualização dos Pesos

Os pesos do modelo são ajustados minimizando a **função de perda**:

Onde:
$$L(\theta) = (Q_{target} - Q(s, a; \theta))^2$$

- $Q_{target} = r + \gamma * \max_{a'} Q(s', a', \theta_{target})$

A perda é utilizada para atualizar os pesos da rede principal através de **backpropagation**, ajustando o modelo para tomar decisões mais precisas no futuro.

Quando o treino convergir, o agente seguirá a política ótima, escolhendo sempre a ação que maximiza o valor $Q(s,a;\theta)$: $\pi(s) = \arg \max_a Q(s, a; \theta)$

Implementação de um Agente de Aprendizado por Reforço para o Jogo da Cobra

Para implementar os conceitos abordados, utilizamos o jogo da cobra, pois este é uma forma fácil e intuitiva de compreender as ideias apresentadas anteriormente.

Inicialmente, desenvolvemos o jogo, cujo código se encontra no ficheiro game.py. Neste ficheiro, criámos toda a interface do jogo, incluindo a cobra, o campo, a comida e as condições para a atribuição de recompensas. Além disso, implementámos condições especiais, como na função “play_step”, onde definimos as ações da cobra. Nesta função, se a cobra não conseguir obter nenhuma recompensa por um período prolongado, o jogo encerra automaticamente, e o agente (neste caso, a cobra) recebe uma recompensa negativa.

Outra função fundamental no desenvolvimento foi a “_move”. Como o objetivo é implementar inteligência artificial no jogo, os movimentos da cobra não podem ser realizados manualmente. Assim, definimos os movimentos com base em três vetores:

- **[1, 0, 0]**: a cobra continua na mesma direção;
- **[0, 1, 0]**: a cobra vira à direita;
- **[0, 0, 1]**: a cobra vira à esquerda.

Com essa abordagem, conseguimos estruturar as ações da cobra de forma a integrar o sistema de inteligência artificial e as regras previamente definidas.

De seguida implementamos o ficheiro agent.py. Este código implementa um agente de **Deep Reinforcement Learning** para jogar o jogo da cobra. Ele utiliza uma rede neural para estimar o valor Q das ações possíveis, aplicando o algoritmo **Deep Q-Learning**. Abaixo, descrevemos as principais seções do código.

Parâmetros Globais

- **MAX_MEMORY**: Define o tamanho máximo da memória utilizada pelo agente para armazenar experiências passadas.
- **BATCH_SIZE**: Número de experiências usadas para treinar a rede neural em uma única iteração.

- **LR**: Taxa de aprendizado da rede neural.

A classe **Agent** é responsável pela implementação do agente que toma as decisões no jogo e treina o modelo para melhorar o seu desempenho. Nesta classe, são inicializadas as seguintes variáveis principais:

- **n_games**: Controla o número de jogos jogados.
- **epsilon**: Gere o equilíbrio entre ações aleatórias (exploração) e ações baseadas no modelo (exploração guiada).
- **gamma**: É o fator de desconto, que prioriza recompensas futuras ao longo de recompensas imediatas.
- O agente também utiliza uma memória limitada para armazenar experiências passadas, implementada como uma deque com um limite máximo (**MAX_MEMORY**).

Além disso, é criado um modelo de rede neural, **Linear_QNet**, que possui:

- **11 entradas**: Representam o estado atual do jogo.
- **256 neurónios na camada oculta**.
- **3 saídas**: Correspondem às ações possíveis que a cobra pode tomar (frente, direita ou esquerda).
- Por fim, o treinador (**QTrainer**) aplica o algoritmo **Q-Learning** para ajustar os pesos da rede e melhorar o desempenho do agente.

Funções principais da classe **Agent**

1. **get_state(game)**

Esta função extrai o estado atual do jogo, representado por **11 variáveis binárias**:

- **3 variáveis para os perigos à frente, à direita e à esquerda**.
- **4 variáveis para a direção atual da cobra**.
- **4 variáveis para a posição da comida em relação à cabeça da cobra** (acima, abaixo, à esquerda ou à direita).

Estas informações são convertidas para um array numérico e usadas como entrada para a rede neural.

2. **remember(state, action, reward, next_state, done)**

Armazena as experiências do agente na memória. Cada experiência inclui o estado atual, a ação tomada, a recompensa obtida, o próximo estado e uma indicação de fim de jogo.

3. **train_long_memory()**

Realiza o treino da rede neural utilizando experiências armazenadas na memória:

- Se o número de experiências for maior que o tamanho definido pelo **BATCH_SIZE**, é feita uma amostragem aleatória.
- Caso contrário, utiliza todas as experiências disponíveis. Este treino contribui para a melhoria do modelo a partir de dados históricos.

4. **train_short_memory(state, action, reward, next_state, done)**

Realiza o treino imediato da rede neural, utilizando apenas a experiência mais recente do jogo. Isto permite ajustes rápidos e incrementais no modelo.

5. **get_action(state)**

Decide qual ação o agente irá tomar com base no estado atual, utilizando o algoritmo **Epsilon-Greedy**:

- Com uma probabilidade controlada por **epsilon**, o agente escolhe uma ação aleatória (exploração).
- Caso contrário, utiliza a rede neural para prever a melhor ação (exploração guiada).

O resultado é uma das três ações possíveis: avançar, virar à direita ou virar à esquerda.

Função **train()**

Esta função constitui o **loop principal de treino do agente**, que executa as seguintes etapas:

1. **Inicialização:**

- É criada uma instância do agente e do jogo.
- São definidas listas para monitorizar a pontuação de cada jogo e a pontuação média ao longo do tempo.

2. **Ciclo de treino:**

- Obtém-se o estado inicial do jogo (**state_old**).
- O agente calcula a ação a ser tomada (**final_move**) com base no estado atual.
- A ação é executada, e o jogo devolve:
 - A recompensa associada.
 - Um indicador de fim de jogo (**done**).
 - A pontuação atual.
- Obtém-se o próximo estado (**state_new**).
- O modelo é treinado com a memória curta (apenas a experiência mais recente).
- A experiência é armazenada na memória longa para treino futuro.

3. **Fim de jogo:**

- Quando o jogo termina, o agente:
 - Reinicia o jogo.
 - Incrementa o número total de jogos jogados (**n_games**).
 - Treina o modelo com as experiências acumuladas na memória longa.
- Caso a pontuação atual seja superior ao recorde anterior, o modelo é guardado como o melhor.

4. **Visualização dos resultados:**

- São gerados gráficos que apresentam as pontuações obtidas em cada jogo e a evolução da pontuação média, permitindo avaliar a performance do agente ao longo do treino.

O ficheiro **model.py** contém a implementação do modelo, composta pela classe **Linear_QNet**, que define a rede neuronal, e pela classe **QTrainer**, que implementa o mecanismo de treino baseado no algoritmo **Q-Learning**.

Classe **Linear_QNet**

A classe **Linear_QNet** define a estrutura da rede neuronal utilizada para estimar os valores Q (*Q-values*). Esta rede é composta por duas camadas lineares:

- **linear1**: Camada linear que conecta a camada de entrada (*input layer*) à camada oculta (*hidden layer*).
- **linear2**: Camada linear que conecta a camada oculta à camada de saída (*output layer*).

Função **forward**

A função **forward** aplica a ativação **ReLU** à saída da primeira camada linear e devolve a estimativa dos *Q-values* para cada ação possível. Esta função permite que o modelo processe o estado do ambiente e produza os valores Q correspondentes às ações disponíveis.

Função **save**

A função **save** guarda os pesos do modelo treinado num ficheiro com o nome **model.pth**. Caso a pasta de destino não exista, é criada automaticamente. Este método assegura a reutilização do modelo em sessões futuras de treino ou teste.

Classe **QTrainer**

A classe **QTrainer** implementa a lógica de treino baseada no algoritmo **Deep Q-Learning (DQN)**.

Variáveis e Componentes

- **gamma**: Fator de desconto, utilizado para priorizar recompensas futuras.
- **lr**: Taxa de aprendizagem, que controla o ajuste dos pesos durante o treino.
- **model**: Instância da rede neuronal **Linear_QNet**.

- **optimizer**: Otimizador **Adam**, utilizado para minimizar a função de perda e ajustar os pesos do modelo.
- **criterion**: Função de perda baseada no **Erro Quadrático Médio (MSE)**, que mede a diferença entre os valores Q previstos e os valores Q-alvo.

Função **train_step**

A função **train_step** realiza um passo de treino com base numa experiência fornecida (estado atual, ação tomada, recompensa recebida, próximo estado e indicador de fim do jogo). O processo é descrito em várias etapas:

1. **Conversão para tensores**:
 - Os estados, ações, recompensas e próximos estados são convertidos para tensores de tipo **float** ou **long**, dependendo do contexto.
2. **Ajuste do formato**:
 - Caso os dados sejam unidimensionais (uma única experiência), são ajustados para corresponder ao formato esperado pela rede neuronal.
3. **Predição dos Q-values**:
 - A rede neuronal calcula os valores Q estimados para o estado atual.
4. **Cálculo dos valores Q-alvo**:
 - Os Q-values alvo são calculados com base na fórmula do **Q-Learning**: ■ Se o jogo não terminou: $Q_{novo} = recompensa + \gamma * \max(Q_{s'})$ ○ O valor correspondente Q_{novo} à ação=tomadarecompensaé atualizado nos valores Q-alvo.
 - Caso contrário:
5. **Minimização da função de perda**:
 - A diferença entre os valores Q previstos e os valores Q-alvo é calculada utilizando a função **MSELoss**.
 - O gradiente é calculado e os pesos do modelo são ajustados com base nesse gradiente, utilizando o otimizador **Adam**.

Observação em Tempo Real

Para uma melhor compreensão dos resultados apresentados nesta seção, recomenda-se executar o projeto e observar, durante algum tempo, como o agente aprende em tempo real.

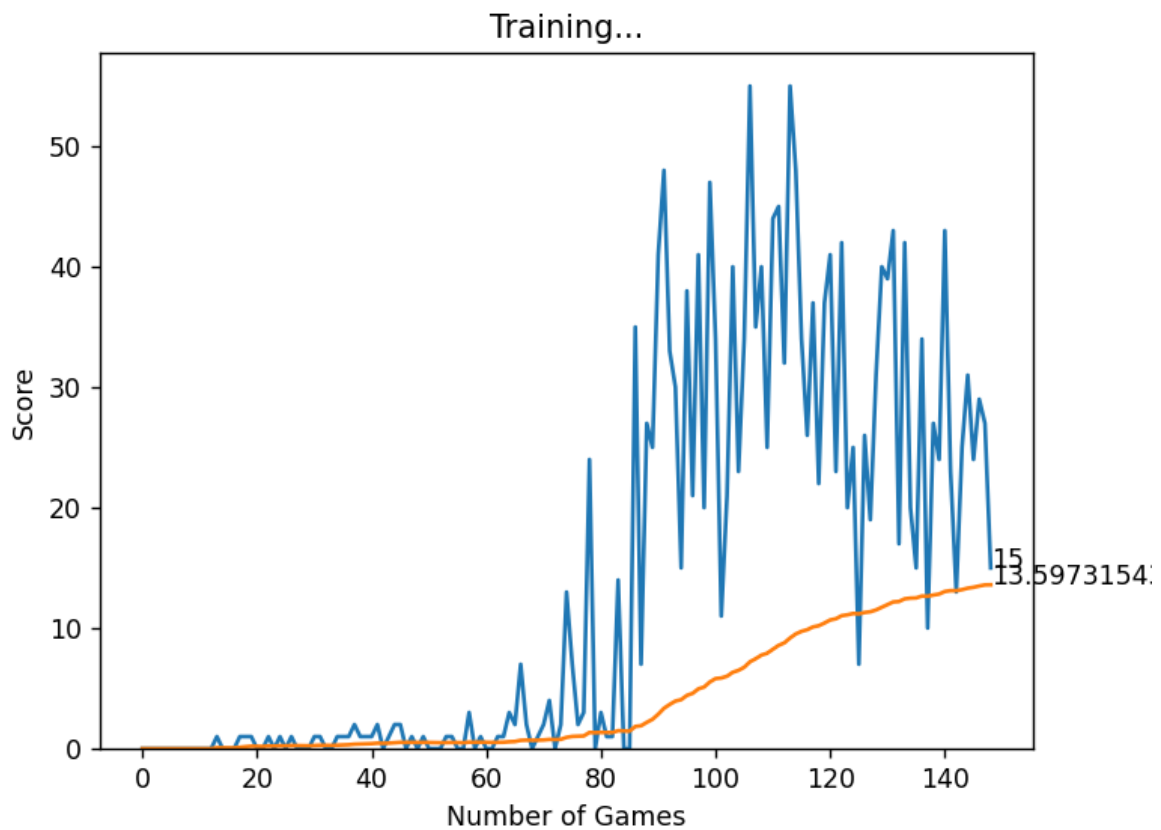
Desempenho do Agente

Após o processo de treino, o agente demonstrou a capacidade de navegar no ambiente de forma eficiente, atingindo uma pontuação superior a 10 em aproximadamente 100 jogos. No início, o agente exibiu um comportamento aleatório, resultado de uma fase inicial de exploração elevada, caracterizada pela estratégia **ϵ -greedy**. À medida que o treino progredia, o agente começou a identificar padrões úteis.

- Inicialmente a cobra faz movimentos complementamente aleatórios
- Após cerca de **10 jogos**, o agente já conseguia evitar colisões com as paredes e consigo próprio. No entanto, o agente ainda colide bastante e parece ter muitos movimentos aleatórios.
- Ao fim de **50 jogos**, parece que o agente se focou em apanhar alimentos de forma mais eficiente. No entanto ele ainda parece muito desorientado, maior parte dos jogos ele acaba por andar às voltas até o jogo dar exit por tempo.
- A partir dos **100 jogos** podemos ver que ela já segue movimento mais retos em direção ao alimento, e acaba apenas por morrer quando colide consigo própria, quanto maior o tamanho da cobra mais dificuldade ela enfrenta a movimentar-se no ambiente. No entanto é a partir desta etapa que começamos a ver o melhor desempenho da cobra, e onde ela acaba por começar a obter resultados acima dos 50,
- Testamos o agente até aos **300 jogos** o que observamos foi que apesar da cobra não passar da pontuação de 60 ela aprende a sobreviver durante muito mais tempo em média.

A curva de aprendizagem evidenciou um aumento consistente na pontuação média, indicando que a abordagem **Deep Q-Network (DQN)** foi eficaz na modelação de uma política otimizada para o ambiente. Contudo, o desempenho do agente apresentou variações em cenários de maior complexidade, especialmente em situações que exigiam decisões rápidas em estados limitados.

Podemos observar os resultados obtidos a partir da análise do gráfico abaixo:



Análise de Estratégias

Durante a análise do comportamento do agente, identificaram-se algumas estratégias básicas que este desenvolveu para maximizar o desempenho:

- **Evitar colisões diretas** com obstáculos, como paredes ou o próprio corpo.
- **Manter uma proximidade controlada** do alimento.
- **Priorizar caminhos** que proporcionem maior liberdade de movimento.

No entanto, em fases mais avançadas do jogo, com o aumento do tamanho da cobra, o agente revelou dificuldades ocasionais devido à incapacidade de prever as consequências de longo prazo das suas ações.

Limitações e Propostas de Melhoria

Embora o modelo tenha apresentado resultados promissores, algumas limitações foram identificadas:

- **Desempenho sub ótimo em estados complexos:** Em situações de alta densidade, onde o número de opções disponíveis era reduzido, o agente apresentou decisões menos eficazes.

- **Exploração insuficiente na fase final do treino:** A redução da exploração (ϵ) pode ter limitado a aprendizagem de estratégias mais diversificadas.

Para mitigar estas limitações podemos propor as seguintes melhorias:

1. **Implementação de Double DQN:** Esta técnica pode aumentar a estabilidade e precisão do modelo ao reduzir a superestimação dos valores de Q.
2. **Utilização de redes neurais convolucionais (CNNs):** Estas redes são mais adequadas para processar entradas visuais complexas, como estados representados por imagens.
3. **Ajuste dinâmico do parâmetro ϵ :** Um mecanismo adaptativo para equilibrar exploração e exploração (exploration-exploitation) poderá melhorar o desempenho global do agente.

Estas propostas visam melhorar a robustez do modelo, permitindo uma maior generalização em cenários de maior complexidade e melhor desempenho em ambientes variados.

Conclusão

Neste projeto, tivemos a oportunidade de explorar uma aplicação de inteligência artificial, especificamente o Reinforcement Learning, num contexto simples, mas bastante desafiador, particularmente, o jogo da cobra. Durante o desenvolvimento, enfrentamos desafios que exigiram de nós a implementação da lógica do jogo e o treino de um agente capaz de aprender com suas interações no ambiente. Esse processo permitiu-nos compreender melhor conceitos como recompensas, exploração e políticas ótimas são essenciais para a construção de agentes inteligentes.

Decidimos utilizar o algoritmo Q-Learning combinado com redes neurais profundas (Deep Q-Network), e essa abordagem mostrou ser eficaz para ensinar a cobra a maximizar as recompensas.